Enabling Data Privacy in Blockchain

Senthilnathan Natarajan
IBM Research

Manish Sethi
IBM Research

Dave Enyeart
IBM Research

Yacov Manevich
IBM Research

Artem Barger
IBM Research

ABSTRACT

Blockchain technology provides a decentralized and immutable ledger, but its inherent transparency poses a significant barrier to enterprise adoption in domains requiring data privacy. Existing solutions like channels provide data isolation but are coarse-grained, requiring a separate ledger and consensus instance per channel. In this paper, we introduce *private data collections*, a mechanism for managing private data on permissioned blockchains. Our approach decouples private data storage from the replicated ledger: private data is shared only among authorized peers, while cryptographic hashes are recorded on the ledger for tamper-evidence and serializability verification. Central to the design is *blind validation*: all nodes can verify the correctness of private transactions using only the on-chain hashes, without ever seeing the private data itself. We design a two-phase gossip-based protocol for proactive dissemination and reactive retrieval of private data, and we formalize the collection abstraction with seven design requirements. We implement our solution within Hyperledger Fabric, leveraging its simulate-order-validate-commit architecture, and demonstrate that private data collections add minimal overhead to transaction throughput while providing fine-grained data privacy. Hyperledger Fabric has shipped the mechanism in production since v1.2, with continued enhancements through v2.5.

PVLDB Reference Format:
Senthilnathan Natarajan, Manish Sethi, Dave Enyeart, Yacov Manevich, and Artem Barger. Enabling Data Privacy in Blockchain. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Enterprise blockchain networks are redefining organizational collaboration by establishing trust without a central intermediary. In sectors ranging from food safety [17, 34] and pharmaceuticals [24] to global trade [20] and mining [19], these systems utilize a permissioned, decentralized ledger to provide a *single source of truth* for high-stakes workflows—reducing audit times from days to seconds and ensuring compliance with regulatory frameworks such as the U.S. Drug Supply Chain Security Act and the EU General Food Law.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Yacov Manevich is now at Ava Labs. Artem Barger is now at Modelyo.

The value of these deployments stems from the blockchain's core properties: a transparent, immutable, and tamper-evident audit trail shared among mutually distrusting parties.

However, a fundamental tension exists between this transparency and the privacy requirements of enterprise data. Sensitive information—such as negotiated pricing, HIPAA-protected medical records [15], or GDPR-governed identities [10]—cannot be fully replicated across a ledger where it is exposed to every participant, including competitors. Yet this data cannot simply be kept off-chain: when trading partners dispute agreed pricing or delivery terms, the blockchain's immutable record must serve as a neutral arbiter that neither party can retroactively alter. Keeping such data entirely off-chain forfeits this dispute-resolution guarantee and the tamper-evident proof required by regulators (e.g., the FDA or SEC). This creates a *privacy-auditability* deadlock: participants must prove the validity of private transactions to the network without revealing the underlying sensitive data.

Resolving this deadlock requires two properties that are individually straightforward but difficult to combine: *fine-grained privacy*, where private data remains visible only to authorized parties, and *verifiable auditability*, where on-chain cryptographic evidence allows any participant to verify integrity and trace provenance. In practice, achieving both on a shared ledger imposes additional constraints. A single business transaction frequently updates both public state and private data—a shipment record and its private invoice, for example—so private data updates and their on-chain cryptographic commitments must commit or abort atomically; a partial commit leaves the on-chain evidence inconsistent with reality. Furthermore, because privacy rules are deeply tied to business logic (who may read a price depends on the contract governing the trade), enforcement must be embedded in the same general-purpose smart contracts—a separate privacy mechanism that restricts expressiveness or imposes prohibitive cost defeats the purpose. Table 1 evaluates existing approaches against these requirements.

Table 1: Comparison of Privacy Mechanisms

Mechanism	Fine-Grained Privacy	Cross-State Atomicity	Smart Contract Access Control	Hardware Agnostic
Channels	×	×	✓	✓
ZKPs	✓	✓	Limited[1]	✓
TEEs (e.g., SGX)	✓	✓	✓	×
Off-chain DB	✓	×	×	✓
Pvt. Collections	✓	✓	✓	✓

[1] ZKP-based smart contracts require expressing all business logic as arithmetic circuits or constraint systems, which limits expressiveness compared to general-purpose languages. Although recent advances in zkVMs are reducing proof times, the per-proof computational overhead remains prohibitive for high-throughput enterprise workloads requiring general-purpose smart contracts.

As the table shows, no existing approach satisfies all four requirements. *Channels* [1] lack fine-grained privacy: each channel is an entirely separate ledger, so every bilateral data-sharing relationship requires its own channel with its own consensus instance, scaling poorly. They also lack cross-state atomicity, since atomic transactions spanning data on separate channels require fragile cross-ledger coordination. *Zero-knowledge proofs* [2, 21] provide fine-grained privacy and atomicity but restrict smart contract expressiveness to arithmetic circuits or constraint systems, with per-proof overhead that remains prohibitive for high-throughput enterprise workloads. *Trusted Execution Environments* [7, 32] satisfy the privacy, atomicity, and smart contract requirements but are not hardware agnostic—they depend on specialized processors with well-documented side-channel vulnerabilities. Off-chain databases offer fine-grained privacy without hardware dependencies but break cross-state atomicity—a shipment record and its private invoice cannot be committed atomically across a blockchain and an external store without fragile two-phase commit—and place private data outside smart contract governance.

This paper presents *Private Data Collections* (PDCs), a native mechanism for achieving fine-grained privacy and cross-state atomicity within a single blockchain ledger. The core of our approach is *blind validation*: by decoupling the storage of private plaintext from its cryptographic commitment, the entire network can verify the serializability and integrity of transactions without accessing the underlying data. Authorized peers process private data in plaintext and concurrently record its hash on the replicated ledger; at commit time, any node can validate state conflicts by comparing hash versions alone. This architecture preserves transactional atomicity across public and private states within a single shared ledger—eliminating the management overhead of channels, the high computational cost of ZKPs, and the hardware dependencies of TEEs. However, excluding private data from the consensus-driven block delivery introduces a critical data availability challenge: ensuring that all authorized peers obtain the required private state before block commitment. We address this with a robust dual-path dissemination protocol: a proactive gossip-based mechanism that propagates data during transaction simulation, complemented by a reactive pull-based recovery for nodes recovering from network partitions or dynamic membership changes.

The main contributions of this paper are as follows:

- We formalize the private data collection abstraction, identify seven design requirements (atomicity, hash-data integrity, smart contract access control, authorized execution, dynamic membership, data lifecycle management, and data availability), and analyze replication protocols to determine their suitability for private data (§3).
- We design a dual-layered storage architecture (§4) built on a *blind validation* mechanism, and formally prove that on-chain hashes serve as a conflict-complete proxy for private state, enabling network-wide serializability verification without exposing private data.
- We propose a two-phase gossip-based protocol (§5) that combines proactive dissemination with reactive retrieval, handles dynamic membership changes, and tolerates network partitions.
- We shipped private data collections in Hyperledger Fabric, where the feature has been production-ready since v1.2 with continued enhancements through v2.5; organizations including Walmart, MineHub, GSBN, MediLedger, Honeywell, and Oracle deploy it today.

2 MOTIVATION FOR DATA PRIVACY

Real-world blockchain deployments span diverse domains—supply chains, healthcare, trade finance—where organizations must share data with relevant parties for coordination while simultaneously protecting it from unauthorized participants and maintaining auditability for the entire network. The dual requirement—*share with a subset, auditable by all*—is the defining challenge. We illustrate it through food-supply-chain traceability, a domain where the tension between privacy and auditability is especially acute.

Food Traceability. In 2006, a major *E. coli* outbreak linked to bagged spinach infected 199 people across 26 U.S. states [28]. Regulators took two weeks to trace the contamination source, during which significant quantities of safe food were wasted [13]. A blockchain-based traceability system addresses this by providing a tamper-evident, shared record—Walmart demonstrated that blockchain could reduce backtrace time from seven days to 2.2 seconds [17, 34]. However, not all data in a food supply chain has the same visibility requirements. Invoices, pricing terms, and discount schedules are exchanged strictly between a buyer and a seller—they must remain private from other participants in the network, including competitors transacting with the same partners.

Why private data must still be on-chain. Yet these records cannot simply be kept off-chain. When a buyer and seller later dispute the agreed price or delivery terms, the blockchain's immutable record serves as an authoritative witness that neither party can retroactively alter. Beyond conflict resolution, when an outbreak triggers an FDA investigation and companies must disclose shipment records, the regulator—and indeed any consortium member—must be able to verify that the disclosed records are genuine and unaltered, even if they never held the private data. On-chain cryptographic evidence makes this possible: the immutable commitment acts as a cryptographic anchor, allowing a regulator to verify a disclosed record against the ledger without the network ever having stored the plaintext—retroactive fabrication becomes impossible.

Atomicity across public and private state. Updating a public shipment record and its private invoice must succeed or fail together—if one commits without the other, the on-chain hash no longer reflects reality and the audit trail becomes untrustworthy. Food standards authorities [9, 11, 12] mandate detailed record-keeping, yet much of it is commercially sensitive. Organizations need fine-grained control over what is shared with whom, without sacrificing the auditability that makes the traceability system trustworthy.

Beyond the coarse question of *whether* data is shared, real deployments require control over *which fields* are shared.

Field-Level Privacy Within a Single Transaction. Privacy requirements vary even within a single transaction. In a food supply chain, a shipment record contains fields with different visibility needs: lot numbers, timestamps, and product origins must be shared across the network for traceability, while prices, discounts, and invoice details are confidential between the buyer and seller. This

private data still needs to be *shared* with the authorized parties—the buyer, the seller, and potentially an auditor—and it must be governed by the same transactional guarantees as the public shipment record. Relegating it to an external database forces organizations to manage a *shadow* coordination layer—replicating data to authorized parties, enforcing access revocation, and maintaining consistency with the on-chain state—all independently of the blockchain, duplicating operational overhead that the blockchain node already subsumes. The alternatives are equally unsatisfying: hiding the entire transaction off-chain sacrifices the audit trail; publishing everything sacrifices business confidentiality; pushing private fields to an external store sacrifices atomicity and adds operational complexity. Integrating privacy into the ledger's native state machine is the only way to guarantee atomicity while eliminating this redundant infrastructure. Fine-grained, field-level privacy *within the blockchain* is what real deployments require.

Privacy as a Participation Prerequisite. Beyond regulatory compliance, transparency creates an economic barrier to participation. Consortium members are often competitors, and blockchain's value increases with participation—network effects are central to its appeal—yet transparency gives organizations a rational incentive to withhold private data or avoid joining altogether. The audit guarantee is what makes organizations willing to join despite privacy concerns: each member trusts that on-chain evidence protects the integrity of every transaction, even transactions involving private data they cannot see. Privacy is therefore not merely a compliance requirement; it is a prerequisite for consortium formation, and auditability is the trust mechanism that enables it.

3 PRIVATE DATA COLLECTIONS: CONCEPT AND DESIGN REQUIREMENTS

Rather than relegating private data to an external store (§2), we integrate it directly into the blockchain node, giving rise to the *private data collection* mechanism.

3.1 Key Idea

The mechanism rests on two complementary pillars: (1) *decoupled storage*, where private data is shared and stored only on peers owned by member organizations of a *collection* with defined membership—ensuring *privacy*; and (2) *on-chain evidence*, where the cryptographic hash of private data is recorded on the replicated ledger—ensuring *auditability*. A single transaction may modify both public and private data simultaneously, and the two must commit atomically; a partial commit renders the on-chain evidence meaningless.

Formally, a private data collection is defined as a tuple $C = (id, \mathcal{M}, \tau)$ with the following parameters:

- id: unique collection identifier
- $\mathcal{M}$: membership policy specifying the set of authorized organizations
- τ: block-to-live (BTL) retention—the number of blocks after which private data is purged, enabling compliance with data-retention regulations (R6); $\tau = 0$ means indefinite retention

A smart contract may define multiple collections, each with independent policies. Additional implementation-specific parameters—such as authorization policies governing write access and dissemination thresholds—are discussed in §4.

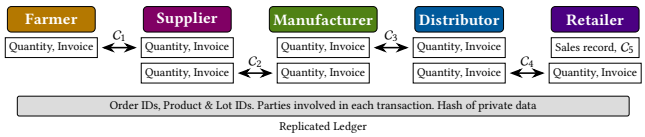

Figure 1: Private data is visible only to authorized parties; on-chain hashes provide proof of existence and traceability.

Figure 1 illustrates the mechanism in the food traceability setting. Five collections (C_1–C_5) are defined along participant relationships: C_1–C_4 each span two adjacent parties in the supply chain, while C_5 is private to the Retailer alone. Public data—order IDs, product lots—lands on the replicated ledger alongside a hash for each private data element; the actual quantities, invoices, and pricing terms are stored only in the relevant collection. Competitors can verify product provenance without ever seeing each other's contracts, and any participant can confirm a disclosed record's authenticity by checking it against the pre-existing on-chain hash. Because each hash is committed as part of a block, the ledger provides a tamper-evident timeline that all participants—even those that never held the private data—can trust.

3.2 Design Requirements

The use cases in §2 and the architectural constraints of our key idea give rise to seven design requirements, across three categories.

Correctness. Integrating private data into a blockchain must not weaken the transactional guarantees the platform is built on. As the food traceability scenario illustrates, updating a public record and its private counterpart must be atomic—a partial commit leaves the ledger inconsistent and the on-chain hash meaningless. Beyond atomicity, the hash recorded on the ledger must faithfully reflect the off-chain data at all times; otherwise, the auditability guarantee evaporates and any party examining the ledger is working from a lie.

R1 *Transactional Atomicity and Isolation:* When a transaction modifies both public and private data, the system must guarantee atomicity and serializable isolation to maintain overall consistency.

R2 *Hash-Data Integrity:* The hash of the private data stored on the replicated ledger must always remain in sync with the actual private data stored in the off-chain collection, acting as a verifiable proof of existence.

Access Control. Privacy guarantees are only as strong as the enforcement beneath them. In a food supply chain, pricing data must be accessible only to the buyer and seller, and access control must be embedded in the same smart contract that governs supply-chain workflows—a separate policy layer creates a seam that can be misconfigured or bypassed. Equally important: private data must never reach unauthorized peers, even transiently during transaction execution.

R3 *Smart Contract Controlled Access:* Private data in a collection must only be accessible or modifiable via a smart contract, ensuring all data interactions adhere to agreed-upon business processes.

R4 *Authorized Execution Only:* Transactions involving private data must only be executed on peers owned by member organizations of the specified collections, preventing unauthorized peers from seeing or processing private data.

Operational. Consortia are not static. Organizations join and leave, regulations change, and data that must be retained today may need to be purged tomorrow. A new carrier joining GSBN needs access to historical shipping documents; an organization subject to data-retention regulations may need private records purged after a mandated period without disturbing the ledger itself. Private data is also excluded from the consensus-driven block delivery by design, so the system must provide an alternative path through which authorized peers can reliably obtain what they are entitled to hold.

R5 *Dynamic Membership:* Collection membership must be mutable. New members must be able to synchronize historical private data, while removed members should lose access to any data persisted after their removal.

R6 *Data Lifecycle Management:* The system must support a block-to-live (BTL) property for private data, allowing for automatic purging to comply with data retention policies or "right to forget" regulations.

R7 *Data Availability:* Since private data is excluded from the consensus-driven block delivery, the system must ensure that all authorized peers can obtain the private data they need before committing a block, even in the presence of network partitions and peer failures.

3.3 Choosing a Blockchain Architecture

To realize these requirements, we must identify a blockchain architecture that natively supports selective execution and data dissemination. In a permissioned blockchain network, a *client* submits transactions on behalf of an application. A *peer* is a node owned by a member organization that maintains a copy of the ledger, executes smart contracts, and validates transactions. An *ordering service* is a decentralized cluster of nodes that establishes the total order of transactions via a consensus protocol. Each peer belongs to exactly one organization, and organizations may operate multiple peers for redundancy. We evaluate three prevalent replication protocols, depicted in Figure 2, to determine which structural properties are necessary for private data support. Throughout this analysis, we assume a crash-fault tolerant (CFT) model for authorized peers; the full threat model is presented in §6.

Order-Execute-Commit and Execute-Order-In-Parallel & Commit. Figures 2(a) and (b) show two protocols that differ in timing: the first orders transactions before execution, the second overlaps both. Despite this structural difference, they share a fatal property for our purposes—every node must execute every transaction to maintain ledger consistency. In order-execute-commit (Ethereum [6], Quorum [26], Hyperledger Besu [18]), blocks are ordered by consensus and then broadcast for serial execution across all nodes. The execute-order-in-parallel variant (BlockchainRDBMS [27]) is more aggressive—peers execute and the ordering service sequences concurrently—but all nodes still verify the final block. Restricting execution to a subset of peers in either model would violate state machine replication (SMR) guarantees, breaking R4 (Authorized Execution Only). Every smart contract runs on every node, which means private data touched during execution is visible to the entire network (violating R3), and without a distinct validation phase there is no mechanism to verify private transactions without re-executing them (violating R7).

Simulate-Order-Validate-Commit. Figure 2(c) presents four phases: simulation (steps 1–2), ordering (steps 3–5), validation (step 6), and commit (step 7). A transaction is speculatively executed on a subset of nodes according to a configurable authorization policy; the execution outcomes are then ordered by a consensus service; finally, all nodes perform validation—including authorization policy checks and optimistic concurrency control (OCC) [22] based serializability checks on the versioned execution outcomes—before committing valid transactions. Hyperledger Fabric [1] implements this protocol. This architecture provides the structural properties needed to satisfy all seven requirements: since transactions are simulated on a subset of nodes, we can restrict simulation to authorized collection members, enabling R4 and R3; the separate validation phase can be augmented to verify hashes of private data on the replicated ledger, enabling R2; the existing OCC-based validation can be extended to cover both public and private state, enabling R1; the decoupling of simulation from consensus creates a natural insertion point for peer-to-peer dissemination, enabling R5 and R7; and because the architecture already separates execution outcomes from the immutable ledger, private data can be stored independently, enabling lifecycle management through purging while retaining on-chain hashes (R6).

The simulate-order-validate-commit protocol is therefore the natural foundation for our design, and the sections that follow explain how we build on it.

4 DESIGN OF PRIVATE DATA TRANSACTIONS

The simulate-order-validate-commit protocol, as argued in the previous section, provides the right foundation for private data. We now describe how private data transaction processing is built on top of it. The design centers on two principles: a *dual-layered storage strategy* that separates raw private data from its cryptographic footprint, and *partitioned state management* that isolates private data while preserving global verifiability. We assume a key-value data model for both public and private state, where each state entry is a $(key, value)$ pair identified by a unique key within a namespace, and each key carries a version that is updated whenever the key is written by a committed transaction. Figure 3 provides an end-to-end overview of the private data collection transaction flow, with numbered steps spanning all four phases; Peer 1 and Peer 2 are members of collection C_1, while Peer n is not. The following subsections detail each component.

4.1 Partitioned State Management

Integrating private data into a shared ledger without exposing it to unauthorized peers during validation is the central challenge. The solution is to partition the state database into three logical domains. Each smart contract is allocated a dedicated *namespace* within these domains; access and modification within a namespace are governed by an endorsement policy specifying the set of authorized peers whose signatures are required to validate state changes. The three partitions for each namespace are:

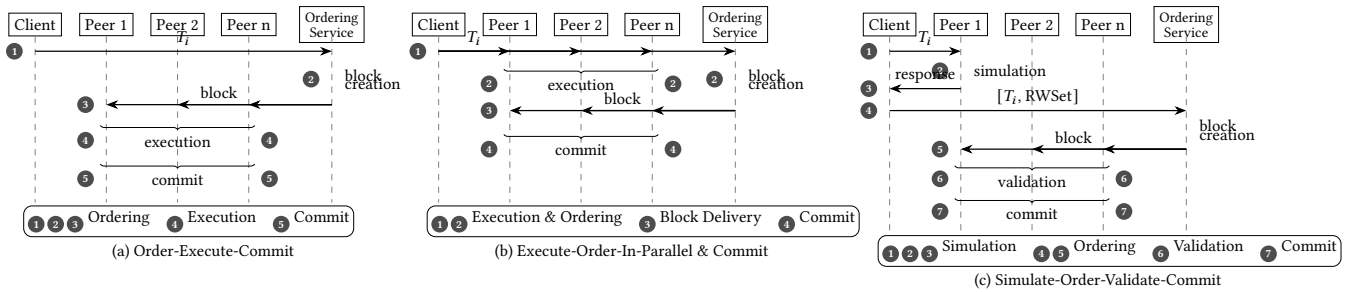

Figure 2: Blockchain replication protocols.

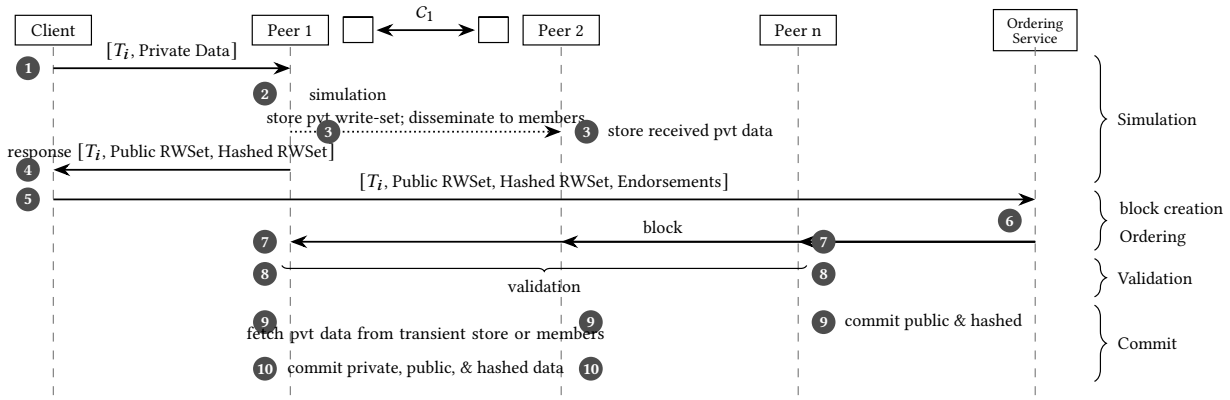

Figure 3: End-to-end private data collection transaction flow. Steps ❶–❹: simulation and dissemination, ❺–❼: ordering, ❽: validation, ❾–❿: commit and retrieval.

- *Public State:* Accessible to all nodes; contains the standard replicated ledger state.
- *Hashed State:* Replicated across all nodes; contains cryptographic hashes of private keys and values, providing the evidence required for tamper-detection and serializability checks.
- *Private State:* Isolated to authorized peers; contains the actual private key-value pairs. Modifications are governed by the collection's own endorsement policy $\mathcal{E}$, which supersedes the smart contract's default endorsement policy for private data operations.

Public State and Hashed State are consistent across all peers in the network, providing a globally uniform view of the ledger. Private State, by contrast, varies from peer to peer depending on which collections each peer's organization belongs to—a peer only maintains private state for collections in which it is authorized. By confining all private data interactions to smart contract namespaces, this partitioning satisfies **R3** (Smart Contract Controlled Access); the replicated Hashed State that mirrors Private State reinforces **R2** (Hash-Data Integrity).

4.2 Transaction Simulation

The simulation phase (steps 1–4 in Figure 3) begins when a client submits a transaction proposal to a subset of peers that are members of the relevant collections. Restricting execution to authorized peers directly satisfies **R4** (Authorized Execution Only). Private data enters the system through a *transient data* field—present during execution but excluded from the final transaction payload, so it is never broadcast to all peers during ordering.

Upon receiving the proposal, each endorsing peer executes the smart contract using the APIs defined in Table 2. Public data operations (`GetState`, `PutState`, `DelState`) access the standard replicated state; private data operations (`GetPrivateData`, `PutPrivateData`, `DelPrivateData`) access collection-specific private state. When a transaction reads and writes both public and private state, the endorsing peers must satisfy both the smart contract's endorsement policy (for public writes) and the collection's policy $\mathcal{E}$ (for private writes). Private writes also perform *dual-recording*: `PutPrivateData`(c, k, v) writes (k, v) to the private write-set and simultaneously records $(H(k), H(v))$ in the hashed write-set, satisfying **R2** (Hash-Data Integrity). Because each endorsing peer independently computes the hash from the private data it receives, the collection's endorsement policy $\mathcal{E}$ bounds the trust assumption: as long as at least one honest endorser participates, the recorded hash faithfully reflects the private data. The implications when all collection members collude are discussed in §6.

On successful execution, the peer produces three partitioned read-write sets: a *public read-write set* for replicated ledger state, a *hashed read-write set* containing cryptographic hashes of private keys and values, and a *private write-set* per collection involved in the transaction. Table 2 illustrates concretely how a sequence of API calls generates these outputs. The endorsing peer signs the public and hashed read-write sets and returns them to the client as part of its endorsement response; the private write-sets, meanwhile,

Table 2: Smart-contract APIs for public and private data operations, with examples illustrating simulation and blind validation.

API	*State Effect*	*Read-/Write-Set Effect*
`GetState`(k)	Returns value of k from public state	Records (k, version) in public read-set
`PutState`(k, v)	—	Records (k, v) in public write-set
`DelState`(k)	—	Records delete of k in public write-set
`GetPrivateData`(c, k)	Returns value of k from collection c	Records ($H(k)$, version) in hashed read-set
`GetPrivateDataHash`(c, k)	Returns $H(v)$ for key k in collection c	Records ($H(k)$, version) in hashed read-set
`PutPrivateData`(c, k, v)	—	Records (k, v) in private write-set; ($H(k)$, $H(v)$) in hashed write-set
`DelPrivateData`(c, k)	—	Records delete of k in private write-set; delete marker in hashed write-set
`PurgePrivateData`(c, k)	Permanently removes k from local storage	Like `Del`, but purges history to comply with "right to forget" policies

Simulation Example. Transaction T_i executes: `GetState`(k_1) → `PutState`(k_1, v_2) → `GetPrivateData`(c_1, k_2) → `GetPrivateData`(c_2, k_3) → `PutPrivateData`(c_1, k_2, v_3) → `PutPrivateData`(c_2, k_3, v_4). This produces three partitioned outputs: **Public RW-Set:** R–$\{(k_1, \text{v1})\}$, W–$\{(k_1, v_2)\}$. **Hashed RW-Set:** R–$\{(c_1, H(k_2), \text{v5}), (c_2, H(k_3), \text{v5})\}$; W–$\{(c_1, H(k_2), H(v_3)), (c_2, H(k_3), H(v_4))\}$. **Private Write-Sets:** c_1: W–$\{(k_2, v_3)\}$; c_2: W–$\{(k_3, v_4)\}$. Here, v1 and v5 denote the version read during simulation.

Validation Example (Blind Validation). Before T_i reaches validation, another transaction T_j commits and updates key k_2 in collection c_1, advancing its hashed state version from v5 to v8. This means T_i's read of k_2 (recorded at version v5 via `GetPrivateData`) is now stale. At validation, the validator—which may not be a member of c_1 and has no access to the private data—performs OCC checks using only the public and hashed state: for k_1, it compares the read version v1 against the current public state version (*match* → pass); for $H(k_2)$ in c_1, it compares the read version v5 against the current hashed state version v8 (*mismatch* → **stale read, conflict detected**); for $H(k_3)$ in c_2, v5 still matches (*match* → pass). Transaction T_i is marked invalid due to the stale read on $H(k_2)$, despite the validator never seeing k_2's actual value—this is blind validation.

are stored in the peer's transient store and proactively disseminated to other collection members as described in §5.

Since transaction order is determined only after simulation completes, committing private data to permanent storage at this point would risk corrupting committed state with transactions that are later invalidated. Peers therefore hold private write-sets in a *transient store* until the transaction commits, satisfying **R1** (Transactional Atomicity and Isolation). Entries are keyed by transaction identifier ($txid$) to prevent collisions among concurrently simulated transactions.

When a smart contract reads private data that is absent from the local store (e.g., due to incomplete dissemination), `GetPrivateData` returns an error and the simulation fails immediately. Only fully informed simulations yield valid endorsements.

4.3 Transaction Validation and Commit

Once the client collects sufficient endorsements, it submits the transaction—containing the public and hashed read-write sets, but not the private write-sets—to the ordering service, which batches transactions into blocks and delivers them to all peers (steps 5–7 in Figure 3). The ordering phase is unchanged from the base protocol; the ordering service never sees private data.

The validation (step 8) and commit (steps 9–10) phases accept only correctly endorsed, serializable transactions. Endorsement policy checks across transactions in a block can run in parallel; OCC validation proceeds sequentially in committed order, since validating one transaction affects the state versions visible to subsequent ones. A serialization graph can alternatively enable parallel validation of non-conflicting transactions [25].

Each transaction's endorsements are checked against both the smart contract's policy (public writes) and the collection's policy $\mathcal{E}$ (private writes), satisfying **R4** (Authorized Execution Only). OCC checks then compare the transaction's read versions against the current Public State and Hashed State. Every write to a private key k atomically updates its hash $H(k)$ in the replicated ledger, so Hashed State version identifiers are sufficient to detect all conflicts on private data without exposing the data itself. *Blind validation* follows directly: every peer can verify serializability of private state changes without ever seeing the private data. Table 2 makes this concrete—a stale read on k_2 in collection c_1 is caught through hashed state version comparison alone, even by a validator with no access to the underlying private values.

We now formalize this correctness claim. We adopt the standard OCC serializability criterion [22]: a transaction T_i is *serializable* with respect to a committed history if no key in T_i's read-set was modified by any transaction T_j that committed after T_i began—equivalently, the read-set versions recorded at simulation time must match the current committed versions at validation time. Let $\mathcal{S}_P$ denote the Private State, $\mathcal{S}_H$ the Hashed State, and $\text{ver}(s, k)$ the version of key k in state s.

Theorem 1 (Conflict-Complete Proxy). *Assuming H is a collision-resistant hash function (so that $H(k_1) = H(k_2) \implies k_1 = k_2$), the Hashed State $\mathcal{S}_H$ is a conflict-complete proxy for the Private State $\mathcal{S}_P$: for any transaction T_i involving private data, an OCC conflict exists in $\mathcal{S}_P$ if and only if a corresponding conflict exists in $\mathcal{S}_H$. Formally, for every private key k in collection c:*

$$\text{ver}(\mathcal{S}_P, k) \neq \text{ver}_{\text{sim}}(T_i, k) \iff \text{ver}(\mathcal{S}_H, H(k)) \neq \text{ver}_{\text{sim}}(T_i, H(k))$$

Proof. We establish three invariants maintained by the API (Table 2):

(1) *Write Bijection:* By construction, every `PutPrivateData`(c, k, v) atomically produces both a private write-set entry (k, v) and a hashed write-set entry $(H(k), H(v))$. The commit procedure assigns both entries the same version identifier (block number, transaction index). `DelPrivateData` and `PurgePrivateData` similarly produce corresponding hashed entries. Thus, $\text{ver}(\mathcal{S}_P, k)$ and $\text{ver}(\mathcal{S}_H, H(k))$ are updated in lockstep: $\text{ver}(\mathcal{S}_P, k) = \text{ver}(\mathcal{S}_H, H(k))$ after every committed transaction.

(2) *Read Recording:* Every `GetPrivateData`(c, k) records $\text{ver}(\mathcal{S}_H, H(k))$ in the hashed read-set. Since Invariant 1 guarantees $\text{ver}(\mathcal{S}_P, k) =$

$\text{ver}(\mathcal{S}_H, H(k))$, this recorded version is identical to the version that would appear in a hypothetical private read-set.

(3) *Conflict Detection:* During OCC validation, a conflict on private key k exists iff $\text{ver}(\mathcal{S}_P, k)$ has changed since simulation. By Invariant 1, $\text{ver}(\mathcal{S}_P, k)$ changes iff $\text{ver}(\mathcal{S}_H, H(k))$ changes. By Invariant 2, the validator compares $\text{ver}(\mathcal{S}_H, H(k))$ against the same baseline. Therefore, the OCC check on $\mathcal{S}_H$ detects a conflict iff the corresponding check on $\mathcal{S}_P$ would.

Since both directions hold, blind validation produces neither false negatives (missed conflicts) nor false positives (spurious aborts). □

Edge Case: Concurrent Membership Changes. A collection's membership policy $\mathcal{M}$ may be updated in the same block that contains private data transactions for that collection. Whether processed sequentially or via a serialization graph [25], a membership change at transaction index j takes effect for all subsequent transactions $j' > j$ within the same block. This within-block ordering is a deliberate design choice: it avoids the complexity of batch-level policy resolution and guarantees deterministic validation across all peers. Endorsement and membership checks during validation therefore always reflect the correct policy version. A newly added member will not yet have historical private data at validation time; the reactive retrieval and reconciliation mechanisms (§5) handle this asynchronously without affecting current-block validity.

Theorem 1 and the sequential policy application ensure **R1** across public and private state.

Once a transaction is successfully validated, the private write-sets are moved from the transient store to the permanent private data store. The Private State within a smart contract's namespace reflects only the latest value for each key, overwritten by each subsequent committed transaction. To support historical retrieval, a separate *private data store* retains all historical private write-sets indexed by block number and transaction index, so that when a new member joins a collection, the complete history of private data can be synchronized to that member.

Handling Missing Private Data. A peer may not possess all private write-sets at commit time. During commit, each peer classifies missing private data into two categories: *eligible missing data*—private data for collections in which the peer's organization is a member but the data has not yet arrived (e.g., due to network delays during dissemination)—and *ineligible missing data*—private data for collections in which the peer's organization is not currently a member. In both cases the peer records the block number, transaction index, collection identifier, and the corresponding hashed key-value pairs from the hashed write-set; the stored hashes allow the peer to verify the authenticity of private data received later during retrieval. Eligible missing entries are fetched via the reactive retrieval protocol described in §5, satisfying **R7** (Data Availability). Ineligible entries are retained because if the organization later joins the collection, it can use them to identify and synchronize historical private data it is now authorized to access, supporting **R5** (Dynamic Membership).

Data Lifecycle Management. The private data store supports data lifecycle management through the collection's block-to-live (BTL) retention parameter τ, satisfying **R6** (Data Lifecycle Management). Any private data committed more than τ blocks ago is considered expired. To comply with privacy regulations such as the "Right to Forget," a background process periodically purges expired private data while preserving the on-chain hashes, keeping the historical audit trail tamper-evident. The implications of retaining these on-chain hashes are discussed in §6.

The design above assumes that private write-sets reach authorized peers before commit time. Ensuring this in practice—and recovering data when it does not arrive in time—is the subject of the next section, which addresses **R5** and **R7**.

5 DATA DISSEMINATION AND RETRIEVAL

Since private data is excluded from the consensus-driven block delivery, we utilize a specialized peer-to-peer protocol to ensure data availability and fault tolerance. We assume a partially synchronous network with potentially malicious clients; the full threat model is presented in §6. The protocol operates in two complementary phases: proactive dissemination and reactive retrieval.

5.1 Proactive Dissemination

As described in Algorithm 1, after simulation the endorsing peer immediately pushes each private write-set to a subset of authorized collection members. Crucially, the endorser does *not* return its signed response to the client until at least $n_{required}$ peers have acknowledged receipt. Blocking on these acknowledgments guarantees that the data exists on multiple peers before the client can submit the transaction for ordering. After sending the endorsement response, a background gossip task continues pushing the write-set to the remaining peers up to n_{dissem} total, spreading coverage without adding to client-observed latency.

We separate $n_{required}$ from n_{dissem} so that operators can tune the two concerns independently. Setting n_{dissem} aggressively ensures most collection members receive the data proactively, which minimizes commit-time reactive pulls; keeping $n_{required}$ small protects endorsement latency.

When the endorser cannot obtain $n_{required}$ acknowledgments before the timeout, it returns a failure to the client, who may retry with different endorsers. We deliberately do not retry dissemination at the endorser: keeping the endorsement critical path simple avoids unbounded retry delays and pushes fault handling to the client, where it belongs.

5.2 Reactive Retrieval

Prior to committing a block, each peer identifies missing authorized private data and retrieves it using the prioritized pull strategy in Algorithm 2. We target the transaction's endorsers first because they are guaranteed to possess the write-sets, having produced them during simulation. Only if the endorsers are unavailable does the peer fall back to broadcasting across the broader gossip network. Each received write-set is verified against the on-chain hash before being stored, so a Byzantine peer cannot inject forged data through this path.

Should retrieval still fail after retries, the peer commits the block anyway and records the missing entries as eligible missing data; a background reconciliation service periodically retries these entries, ensuring eventual availability without blocking block commit.

Algorithm 1 Proactive dissemination protocol.

Proactive Dissemination Protocol

Input: Transaction tx with private write-sets

At Endorsing Node e (after simulation):

1. Store all private write-sets for tx in local transient store
2. **for each** collection c in tx's private write-sets **do**
   - $W_c \leftarrow$ collection c's private write-set
   - *// Blocking phase: wait for $n_{required}$ ACKs before responding*
   - $P_{req} \leftarrow$ SelectAuthorizedPeers$(c.\mathcal{M}, c.n_{required})$
   - Push $(tx.id, c.id, W_c)$ to all $p_i \in P_{req}$
   - Wait until $|\{\text{org}(p_i) : p_i \text{ ACKed}\}| \geq c.n_{required}$, or timeout
   - **if** timeout **then** return dissemination failure to client
3. Return signed endorsement response to client
4. **for each** collection c in tx's private write-sets **do**
   - *// Background phase: spread to remaining peers up to n_{dissem}*
   - $P_{dissem} \leftarrow$ SelectAuthorizedPeers$(c.\mathcal{M}, c.n_{dissem})$ $\setminus P_{req}$
   - Push $(tx.id, c.id, W_c)$ to all $p_i \in P_{dissem}$ (non-blocking)

At Receiving Peer p_i:

1. Verify sender e is authorized for collection c
2. Verify ledger height of tx is within acceptable range
3. Store $(tx.id, c.id, W_c)$ in local transient store
4. Send ACK to endorser e

Algorithm 2 Reactive retrieval protocol.

Reactive Retrieval Protocol

Input: Block B with hashed write-sets, local transient store

At Committing Peer p (before commit):

1. **for each** transaction $tx \in B$ **do**
   - **if** tx fails endorsement policy **then skip**
2. **for each** collection c referenced in tx's hashed write-set **do**
   - **if** $p \in c.\mathcal{M}$ **and** c's private write-set $\notin$ transient store
     - Add digest $(tx.id, c.ns, c.id, B.num, tx.idx)$ to MissingSet
3. **if** MissingSet $= \emptyset$ **then** proceed to commit
4. $E \leftarrow$ endorsers of tx *// priority targets*
5. Send pull request with MissingSet digests to E
6. **for each** private write-set W_c received **do**
   - Verify $H(W_c)$ matches on-chain hash; if valid, store W_c
7. **if** MissingSet $\neq \emptyset$ **then**
   - Broadcast to gossip network; retry with fixed-interval sleep
8. Proceed to commit (missing data tracked for reconciliation)

5.3 Dynamic Membership and Reconciliation

Collection membership may change over the lifetime of a network. When a new member is added to a collection's membership policy $\mathcal{M}$, the peer detects the configuration change during the commit of the block containing the policy update. At that moment, all previously recorded *ineligible missing data* for that collection—entries the peer had tracked but was not authorized to fetch—are reclassified as *eligible missing data*. Reclassifying these entries automatically triggers the background reconciliation service, which periodically pulls the corresponding historical private write-sets from existing collection members using the same hash-based verification as the reactive retrieval protocol.

In this way, a newly admitted peer catches up on all historical private data it is now authorized to access, supporting **R5** (Dynamic Membership).

Membership removal is handled symmetrically. The departing peer retains access to data committed before its removal, but the updated policy excludes it from both proactive dissemination and reactive retrieval eligibility checks, preventing any subsequent private data from reaching that organization.

6 THREAT MODEL AND SECURITY ANALYSIS

Our design assumes a decentralized network of organizations in which some nodes may be faulty or malicious. We characterize the trust assumptions for each entity class and the security implications that follow.

Peer Nodes. We assume a crash-fault tolerant (CFT) model for authorized peers. For *correctness*, the endorsement policy is the trust boundary for hash-data integrity (R2). Each endorsing peer independently computes the hash of private data during simulation; since the endorsement policy requires matching responses from multiple organizations, a single compromised peer cannot record a fraudulent hash. However, if all endorsing members of a collection collude, they can commit a hash that does not correspond to the actual private data, breaking R2. This is an inherent limitation: the system guarantees hash-data integrity only under the assumption that the endorsement policy holds—i.e., at least one endorser is honest. *Confidentiality* is more nuanced. A malicious peer can leak private data it legitimately holds, or refuse to disseminate data to eligible peers; these are inherent limitations of access-control-based privacy, which prevents unauthorized access but cannot constrain how an authorized party uses data it possesses. Confidentiality is therefore bounded by the weakest collection member. Purging offers no stronger assurance: a malicious member can retain private data locally without honoring purge requests, so purge is best-effort rather than a cryptographic guarantee.

Ordering Service. Only public and hashed read-write sets are submitted to the ordering service, so ordering nodes never see private data payloads. The ordering layer supports both CFT ordering via Raft, which tolerates crash failures as long as a majority of nodes remain available, and BFT ordering, which further tolerates Byzantine faults provided fewer than one-third of ordering nodes are malicious.

Clients. Clients may be malicious, submitting inconsistent private data to different endorsers or forging simulation results. The hash mechanism neutralizes the first attack: if a client sends different private data to different endorsers, each endorser computes a different hash and includes it in its signed response. Because the endorsement policy requires matching responses from multiple endorsers, the divergent hashes render the endorsements incompatible and the transaction is rejected during validation.

A subtler attack is write-set flooding: a client simulates many transactions but never submits them, causing orphan write-sets to accumulate in the transient store. We counter this with a retention window Δ—the transient store automatically purges entries whose simulation height lags the current ledger height by more than Δ blocks, and receiving peers reject disseminated write-sets outside this window. Setting Δ requires care: it must exceed the expected

block count between simulation and commit, which depends on network latency and block creation rate; too small a value risks permanently losing private data for valid transactions delayed in the ordering pipeline. In our implementation, Δ is governed by `transientstoreMaxBlockRetention` (default: 1000 blocks), and `pullRetryThreshold` (default: 60 s) bounds how long a peer retries reactive retrieval before recording data as permanently missing. A second attack vector involves fabricated simulations lacking sufficient endorsements; peers counter this by performing endorsement policy validation *before* attempting to pull any missing pre-images, discarding invalid transactions without fetching their data.

Network. We assume a partially synchronous network in which partitions are possible but eventually resolve. A partition may delay private data dissemination, but correctness is not compromised: a peer that cannot obtain private data before commit records it as missing and reconciles once connectivity is restored.

Metadata Leakage. Private data collections protect payload content but not transaction *metadata*. The hashed read-write sets recorded on the public ledger reference specific collections, and collection membership policies are visible to all network participants; any peer can therefore infer which organizations are party to a private transaction, even without seeing the underlying data. Eliminating this leakage would require cryptographic techniques such as zero-knowledge membership proofs [?] or anonymous credentials [?], which let a peer prove membership in an authorized collection without disclosing which collection it belongs to. We leave such extensions as future work.

7 IMPLEMENTATION

The private data collection mechanism described in §4 was built directly into Hyperledger Fabric [1], an open-source permissioned blockchain platform, mapping each architectural component to a peer-internal service.

Fabric Architecture Overview. Fabric's simulate-order-validate-commit paradigm is described in §3. Within a peer, the central components are the *endorser* responsible for chaincode simulation, the *gossip* layer for block dissemination, endorsement policy and serializability validators, and a *committer* that persists valid transactions to the *block store*, *state DB*, and *history DB*. Figure 4 shows how private data collection components (green) extend the existing architecture (white).

Storage Architecture. Supporting the dual-layered storage strategy required extending the peer with four logical stores: (i) a *transient store* for uncommitted write-sets awaiting dissemination and commit; (ii) a *private data store* maintaining the complete history of committed private data for retrieval and reconciliation; (iii) an *eligible missing data* tracker for private data the peer is authorized to access but has not yet received; and (iv) an *ineligible missing data* tracker for collections the peer does not currently belong to, which are reclassified as eligible when collection membership changes.

Hash Function. The hash function H applied to private keys and values is configurable per channel via a configuration transaction. Fabric supports SHA-256, SHA-512, and other SHA-family algorithms, with SHA-256 as the default; all satisfy collision-resistance under standard cryptographic assumptions, meeting the precondition of Theorem 1.

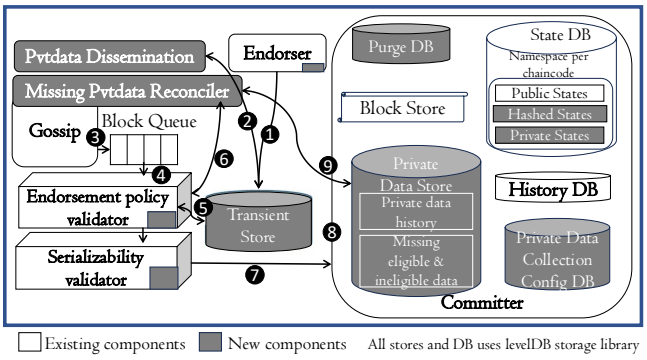

Figure 4: Peer architecture with private data collection components. Pre-existing components (white) form the core transaction pipeline; new components (green) implement the private data collection mechanism.

Partitioned State Management. Fabric's existing State DB was extended with namespace-based isolation to realize partitioned state management. A chaincode's public, hashed, and private state all reside in the same physical database but are distinguished by internal key prefixes; the peer's data management layer enforces that no smart contract can read or write another chaincode's hashed or private namespaces.

Gossip Protocol Extensions. The dissemination and retrieval protocols (§5) were integrated into Fabric's gossip component through new message types: proactive push during simulation (Algorithm 1) and reactive pull during commit (Algorithm 2). A configurable retry mechanism handles transient failures in both paths.

Collection Configuration and Membership. Collection policies are stored as part of the chaincode definition in the channel configuration. Adding a new organization triggers a reclassification of previously ineligible missing-data entries, after which the `reconciliation` service detects the newly eligible entries and fetches the corresponding historical private data on the peer's behalf.

Lifecycle and Purging. Two complementary purging mechanisms are supported. A background retention process within the `PvtDataStore` automatically expires private data based on τ, running in asynchronous batches so as not to impede commit throughput. For application-controlled deletion, the `PurgePrivateData` API (Table 2) commits a purge marker to the ledger; once committed, the marker prevents the data from appearing in query results or being reconciled from remote peers, providing the "right to forget" behavior required by data protection regulations.

Validation and Commit. The validation component was extended to check both chaincode-level and collection-level endorsement policies. Blind validation works by running OCC checks against the hashed namespace in the State DB, so every node can verify serializability without ever seeing the underlying private data.

Implementation Scope. The private data collection feature is written in Go and totals approximately 12,000 lines of production code spread across the peer's endorser, committer, gossip, and state management subsystems. The three largest components are private data storage (~4,000 lines), gossip-based dissemination and retrieval

(~3,300 lines), and the privacy-enabled state database (~1,700 lines). An additional ~21,000 lines of unit and integration tests accompany the feature, a 1.6:1 test-to-production ratio that reflects the importance of correctness in a production storage system.

8 EVALUATION

The design in §4–§5 introduces several mechanisms—dual-recording, hashed-state blind validation, proactive dissemination, and reactive retrieval—each adding work relative to a standard public-only transaction. Our evaluation seeks to answer the following questions:

(1) *What is the overall cost of privacy?* How does end-to-end throughput and latency of PDC transactions compare to identical workloads that store all data on the public ledger?
(2) *Where does the overhead originate?* During simulation, each `PutPrivateData` performs dual-recording and the endorser waits for $n_{required}$ dissemination acknowledgments before responding. During validation, every peer performs OCC checks on both public and hashed state. During commit, peers may need to reactively retrieve missing private data. Which phase dominates the overhead?
(3) *How sensitive is performance to the dissemination parameters?* The endorser targets n_{dissem} peers and blocks until $n_{required}$ acknowledge. Aggressive settings reduce reactive retrieval but increase endorsement latency; conservative settings risk commit-time pulls. What is the optimal trade-off?
(4) *How does the system scale with the number of collections and network size?* More collections per transaction mean more dissemination targets, more hashed-state entries, and more endorsement policy checks. Does throughput degrade gracefully?
(5) *How efficiently can new members reconcile historical data?* When an organization joins a collection, the background reconciliation service must retrieve all historical private write-sets. How does reconciliation time grow with collection history, and does it impact steady-state throughput?
(6) *How resilient is the system to adverse conditions?* Peer failures and network partitions may cause dissemination failures, forcing reactive retrieval. How does throughput degrade under increasing failure rates?

8.1 Experimental Setup

To answer these questions, we design four application workloads from different industry domains, each modeling a realistic multi-party business process on Hyperledger Fabric. The workloads vary along several dimensions—network size, lifecycle complexity, privacy ratio, endorsement strictness, and read-write intensity—to stress different aspects of the PDC mechanism. For each workload, we implement two chaincode variants in Go: a *PDC variant* that stores sensitive fields in private data collections, and an *all-public variant* that stores identical data on the public ledger. Comparing the two isolates the overhead introduced by private data collections. Table 3 summarizes the workload characteristics and experimental setup; all workloads and experiment scripts are open-sourced.[2]

[2]URL redacted for review.

Table 3: Workload characteristics and experimental setup.

	Supply Chain	Auction	Trade Finance	EHR
Domain	Farm-to-retail	Sealed-bid auction	Int'l trade	Patient care
Orgs / Peers	10 / 10	6 / 6	12 / 12	8 / 8
PDC collections	17	9	21	13
Only-PDC tx	—	48.1%	45.9%	0.0%
Mixed tx	—	0.0%	0.0%	48.6%
Public-only tx	—	51.9%	54.1%	51.4%
Endorsement policy	OR (any 1)	AND (2/2)	Majority (7/12)	2-of-4 roles
Reads / Writes	1 / 1	2 / 1	1 / 2	3 / 1
VMs†	14	10	16	12

†Each VM: 32 vCPUs (2x8-core Xeon), 64 GB RAM, 12 TB disk, RHEL 9 / 3 Raft orderers + 1 client per workload. VMs span 4 WAN regions on IBM Research Cloud (E=us-east-1, S=us-south-1, EU=eu-de-1, JP=jp-tok-1); symmetric RTTs (ms): E–S 31, E–EU 85, E–JP 139, S–EU 119, S–JP 170, EU–JP 221. Load is increased until throughput saturates; we report peak tx/s and latency (proposal + broadcast + commit) over 300 s steady-state runs.

Food Supply Chain models a farm-to-retail product lifecycle across five roles (Farmer, Supplier, Manufacturer, Distributor, Retailer) with 17 bilateral collections for private pricing and invoice data, exercising the largest collection count per organization. *Sealed-Bid Auction* models a sealed-bid process where ~48% of transactions are private-data-only and the remainder are public-only, stressing dissemination throughput with a strict AND endorsement policy. *Trade Finance* models cross-border trade across six roles with the longest lifecycle (22 steps) and highest write amplification (2 writes per tx), testing PDC scalability under a majority endorsement policy. *Electronic Health Records* models patient care across four roles; it is read-heavy (3 reads per tx) and uses a 2-of-4-roles endorsement policy, stressing PDC read throughput. In addition, we develop `PDC_BENCH`, a minimal chaincode that exposes four operations—`WritePrivateData`, `ReadPrivateData`, `WritePublicData`, and `ReadPublicData`—enabling controlled measurement of individual PDC mechanism overhead against a public-ledger baseline.

8.2 Commit-Path Overhead and Optimizations

To provide early insight into the cost of privacy, we ran a microbenchmark using `PDC_BENCH` (4 organizations, 4 peers) with all data stored in a single private data collection. Without optimizations, the private data variant achieved only 720 tx/s compared to 1440 tx/s for the public-only variant—a 50% throughput reduction. Profiling the block commit path revealed four new steps that account for the gap: (1) `ListMissingPvtData`, invoked by the gossip coordinator to identify, for every (transaction, collection) pair in the block, whether the peer is a member of the collection and, if so, whether the corresponding private data is already available in the transient store or must be fetched from other collection members; (2) committing to the private data store (`PvtStore`); (3) updating purge entries in `PurgeDB`; and (4) purging consumed entries from the transient store.

We applied five targeted optimizations. First, the per-collection membership check in `ListMissingPvtData` performs a full cryptographic policy evaluation (certificate deserialization, MSP validation, and signature-policy evaluation) for every (transaction, collection) pair, yet this is unnecessary: since collection membership is determined solely by organization identifiers, the crypto check is replaced with a simple string comparison of the peer's MSP ID against the member org list. Second, many commit-path operations are independent and can be parallelized: committing to the block store and `PvtStore` can proceed concurrently, and updating `PurgeDB` and the state DB can run in parallel. Third, transient store purging is deferred to a background goroutine. Fourth, a cache is introduced for the transient store. Each optimization is crash-safe: if the peer fails mid-commit, it recovers by comparing the block store and private data store heights and rolling back. After applying all five optimizations incrementally, throughput recovered from 720 to 1344 tx/s—reducing the overhead from 50% to only 6% relative to public-only transactions. Table 4 summarizes the incremental gains from each step.

Table 4: Incremental effect of commit-path optimizations.

Optimization	tx/s	Gain
No optimizations (PDC baseline)	720	—
+ O(1) membership check	880	+22%
+ Background transient-store purge	1040	+44%
+ Parallel blk store & pvt store	1152	+60%
+ Parallel purge DB & state DB	1248	+73%
+ Transient-store cache	1344	+87%
Public data only (reference)	1440	—

8.3 End-to-End Performance

To answer EQ1, we compare the PDC and all-public variants of all four workloads at peak throughput. Figure 5 visualizes throughput and latency side by side.

For three of the four workloads, the PDC variant incurs a moderate throughput overhead relative to the all-public baseline: 3% for Auction (1,025 vs. 1,059 tx/s), 7.5% for Supply Chain (1,468 vs. 1,587 tx/s), and 14% for EHR (1,021 vs. 1,194 tx/s). The overhead correlates with per-transaction complexity: EHR is the most read-heavy workload (3 reads per tx), and each read of private data requires an additional transient-store or private-data-store lookup, amplifying the PDC cost. Auction and Supply Chain, with simpler read-write profiles, see smaller overheads.

Trade Finance is a notable outlier: the PDC variant is 18% *faster* than the all-public baseline (1,200 vs. 1,018 tx/s). This workload requires 7 endorsements per transaction from a majority (7-of-12) policy, producing the highest endorsement latency across all workloads (~3.5 s). In this regime, the orderer and committer are lightly loaded, so the additional dissemination and hashed-state work fits within the slack created by the long endorsement phase. Meanwhile, PDC transactions write a hash to the public ledger rather than the full payload, reducing the block size and orderer bandwidth, which slightly improves ordering throughput.

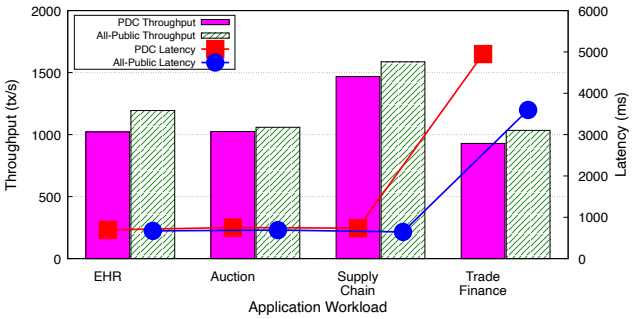

Figure 5: End-to-end throughput (bars, left axis) and latency (lines, right axis) for PDC and all-public variants across four application workloads.

Latency follows a similar pattern: the first three workloads see 3–14% higher latency under PDC, while Trade Finance latency is comparable (3,557 vs. 3,691 ms), confirming that the endorsement phase—not the PDC mechanism—dominates end-to-end latency when the endorsement policy is strict.

8.4 Sensitivity and Scalability

Dissemination Parameter Sensitivity (EQ3).

Scalability with Collections and Network Size (EQ4).

8.5 Dynamic Membership and Fault Tolerance

Dynamic Membership and Reconciliation (EQ5).

Fault Tolerance under Adverse Conditions (EQ6).

9 REAL-WORLD IMPACT

Since its introduction in Hyperledger Fabric v1.2, the private data collection mechanism has seen broad adoption across industries with distinct privacy requirements. MineHub [19] deploys PDCs to isolate bilateral contract data across thousands of supply-chain participants in mining and metals, replacing the coarse-grained channel model that could not scale to dynamic data-sharing relationships. The IBM Food Trust—deployed at Walmart and other major retailers [17, 34]—protects supplier-specific pricing and process data with PDCs while preserving end-to-end traceability for food safety. GSBN (Global Shipping Business Network) [20] relies on PDCs to keep private trade documents such as Bills of Lading confidential within its permissioned shipping network.

The pharmaceutical sector illustrates a particularly instructive use case. The MediLedger project [24] combines PDCs with zero-knowledge proofs for drug tracking under the U.S. Drug Supply Chain Security Act (DSCSA), an approach validated by an FDA pilot program; this confirms that PDCs can serve as a substrate for layering additional cryptographic guarantees when the threat model demands them. In aerospace, Honeywell's GoDirect Trade marketplace [16] uses both channels and private data collections to isolate transaction data in its B2B marketplace for used aircraft parts. In financial services, Oracle's Blockchain Platform [29] stores bank account numbers solely in the private data of participating organizations while recording one-way hashes on the shared ledger,

protecting account attributes in a wholesale Central Bank Digital Currency (CBDC) solution.

Across all of these deployments the underlying pattern is the same: PDCs partition sensitive bilateral data—pricing, contracts, account identifiers—while preserving a shared, hash-verified audit trail that satisfies regulatory compliance and cross-party verification requirements.

10 RELATED WORK

Channels and Sub-Networks. Hyperledger Fabric's *channels* [1] provide data isolation via independent blockchain instances, each with its own ledger and consensus. While effective, channels are coarse-grained: each data-sharing relationship may require a separate channel, leading to operational complexity. Our mechanism provides fine-grained privacy within one channel.

Private Transactions via Off-Chain Managers. Quorum [26] and Hyperledger Besu [18] both implement a hash-on-chain / data-off-chain model using an external transaction manager (Tessera). However, both have static privacy group membership, require all participants to be online, and offer limited interoperability between private and public state within a single transaction. Our mechanism supports dynamic membership with retroactive reconciliation, tolerates offline peers via gossip, and enables atomic cross-state reads and writes. R3 Corda [4] eliminates the global ledger entirely, sharing transactions only on a need-to-know basis. This introduces a *backchain resolution* problem [4]: to verify an input state, a recipient must walk the chain of prior transactions that produced it, and each step in this chain may involve data from unrelated parties who never consented to its disclosure. The result is that verifying provenance can inadvertently expose historical data to new counterparties. That same architectural choice also precludes network-wide queries and the tamper-evident audit trail that our on-chain hashes provide.

Cryptographic Privacy. Zero-knowledge proof systems provide strong privacy guarantees: Zerocash [2] uses zk-SNARKs [3] for fully anonymous payments; Hawk [21] extends this to smart contracts but requires a trusted manager; Zether [5] enables confidential token transfers via ElGamal encryption and range proofs. All incur significant computational cost (seconds per proof), making them unsuitable for high-throughput enterprise workloads. Our work is complementary—MediLedger [24] demonstrates combining PDCs with ZKPs in practice. Confidential Transactions [23], based on Pedersen commitments [30], hide amounts via homomorphic commitments but not sender/receiver identities. CryptoNote [33] (Monero) combines ring signatures and stealth addresses for anonymity. These target permissionless cryptocurrency privacy—a different problem from selective access control in permissioned networks.

TEEs, MPC, and Homomorphic Encryption. TEE-based systems such as Ekiden [7] and CCF [32] execute smart contracts inside hardware enclaves (e.g., Intel SGX), but introduce hardware dependencies and side-channel vulnerabilities. MPC platforms like Enigma [35] enable joint computation over private inputs, but performance overhead led the project to pivot to TEEs. Fully homomorphic encryption [14] allows computation on encrypted data but incurs 10^4–$10^6\times$ overhead. Our approach sidesteps these costs: authorized peers process private data in plaintext while the blockchain stores only hashes, requiring no specialized hardware.

Database-Layer Privacy. CryptDB [31] executes SQL over encrypted data with practical overhead (15–26%) but progressively leaks information. Differential privacy [8] provides statistical guarantees against individual record disclosure. These complement our work: PDCs control *who* accesses data at the replication layer, while database techniques limit *what* queries reveal.

Positioning. The distinguishing property of our mechanism is that it achieves fine-grained privacy through architecture—selective replication, on-chain hashes, and blind validation—rather than through expensive cryptographic primitives. Channels impose isolation at the granularity of an entire ledger instance, while our approach operates within a single shared ledger, avoiding the operational complexity of managing many independent chains. Corda eliminates the global ledger entirely to maximize privacy, but at the cost of backchain resolution and the loss of a network-wide audit trail; our on-chain hashes preserve both verifiability and provenance without exposing payload data. Cryptographic approaches such as ZKPs and homomorphic encryption offer provable confidentiality bounds, but their computational overhead—seconds per proof in the case of zk-SNARKs—is incompatible with high-throughput enterprise workloads, whereas PDCs process private data in plaintext on authorized peers and impose negligible overhead. TEE-based systems achieve similar throughput but introduce hardware trust assumptions and side-channel exposure that many enterprise deployments are unwilling to accept.

11 CONCLUSION

We presented *private data collections* (PDCs): private data is stored only on authorized peers while its cryptographic hash is recorded on the replicated ledger. A transient store, hashed read-write sets, and a gossip-based dissemination protocol together deliver privacy, atomicity, integrity, and auditability within a single shared ledger—without specialized hardware, expensive cryptographic primitives, or the management overhead that makes channel-based deployments brittle at scale. Our Hyperledger Fabric implementation has been production-ready since v1.2 and achieves throughput and latency comparable to standard public transactions.

Deployments at Walmart, MediLedger, GSBN, and MineHub confirm that the design generalizes across industries with very different trust models and data-sharing patterns. More broadly, PDCs demonstrate that selective transparency—giving designated parties verifiable access to exactly the data they are authorized to see—is achievable within a standard permissioned blockchain architecture, opening a practical path for regulated industries that have so far found blockchain's transparency model incompatible with their compliance obligations.

REFERENCES

[1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, pages 30:1–30:15, New York, NY, USA, 2018. ACM.

[2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy (SP)*, pages 459–474. IEEE, 2014.

[3] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *Proceedings of the 23rd USENIX Conference on Security Symposium*, SEC'14, page 781–796, USA, 2014. USENIX Association.

[4] R. G. Brown, J. Carlyle, I. Grigg, and M. Hearn. Corda: An introduction. R3 Technical White Paper, 2016.

[5] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh. Zether: Towards privacy in a smart contract world. *IACR Cryptology ePrint Archive*, 2019:191, 2019.

[6] V. Buterin. Ethereum: A next-generation smart contract and decentralized application platform, 2014. Accessed: July 31, 2019.

[7] R. Cheng, F. Zhang, J. Kos, W. He, N. Hynes, N. Johnson, A. Juels, A. Miller, and D. Song. Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts. In *Proceedings of the IEEE European Symposium on Security and Privacy*, EuroS&P '19, pages 185–200. IEEE, 2019.

[8] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[9] European Commission. Food safety policy. https://ec.europa.eu/food/horizontal-topics/general-food-law_en, 2023.

[10] European Parliament and Council of the European Union. General data protection regulation (GDPR) – regulation (EU) 2016/679, 2016.

[11] Food and Agriculture Organization of the United Nations. Food traceability guidance. http://www.fao.org/3/a-i7665e.pdf, 2017.

[12] Food Standards Australia New Zealand. Food standards Australia New Zealand. https://www.foodstandards.gov.au/, 2023.

[13] J. S. A. G. C. N. Gaur. *Blockchain for Business*. Addison-Wesley Professional, paperback edition, 2019.

[14] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, pages 169–178, New York, NY, USA, 2009. ACM.

[15] HHS. Hipaa privacy rule.

[16] Hyperledger Foundation. Case study: Honeywell aerospace creates online parts marketplace with Hyperledger Fabric. https://www.lfdecentralizedtrust.org/case-studies/honeywell-case-study, 2019.

[17] Hyperledger Foundation. How Walmart brought unprecedented transparency to the food supply chain with Hyperledger Fabric. https://www.hyperledger.org/wp-content/uploads/2019/02/Hyperledger_CaseStudy_Walmart_Printable_V4.pdf, 2019.

[18] Hyperledger Foundation. Hyperledger Besu. https://www.hyperledger.org/use/besu, 2020.

[19] Hyperledger Foundation. Case study: Managing the metal and mining industry's supply chain. https://www.hyperledger.org/case-studies/minehub, 2021.

[20] Hyperledger Foundation. GSBN simplifies global trade with Hyperledger Fabric. https://www.hyperledger.org/case-studies/gsbn, 2022.

[21] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 839–858, 2016.

[22] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2):213–226, June 1981.

[23] G. Maxwell. Confidential transactions, 2015.

[24] MediLedger. DSCSA pilot project report. https://www.mediledger.com/fda-pilot-report, 2020.

[25] H. Meir, A. Barger, and Y. Manevich. Increasing concurrency in hyperledger fabric. In *Proceedings of the 12th ACM International Conference on Systems and Storage*, SYSTOR '19, pages 179–179, New York, NY, USA, 2019. ACM.

[26] J. Morgan-Chase. Quorum private transactions.

[27] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran. Blockchain meets database: Design and implementation of a blockchain relational database. *Proc. VLDB Endow.*, 12(11):1539–1552, July 2019.

[28] National Center for Biotechnology Information. Escherichia coli. https://www.ncbi.nlm.nih.gov/books/NBK507845/, 2023.

[29] Oracle. Robust, scalable and efficient system for confidential token transactions: Oracle's breakthrough on Hyperledger Fabric. https://blogs.oracle.com/blockchain/robust-scalable-and-efficient-system-for-confidential-tokenized-transactions-oracles-breakthrough-on-hyperledger-fabric, 2024.

[30] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In J. Feigenbaum, editor, *Advances in Cryptology — CRYPTO '91*, pages 129–140, Berlin, Heidelberg, 1992. Springer Berlin Heidelberg.

[31] R. A. Popa, C. M. S. Redfield, N. Zeldovich, and H. Balakrishnan. Cryptdb: Protecting confidentiality with encrypted query processing. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, SOSP '11, pages 85–100, New York, NY, USA, 2011. ACM.

[32] M. Russinovich, E. Ashton, C. Avanessians, M. Castro, A. Chamayou, S. Clebsch, M. Costa, C. Fournet, M. Kerner, S. Krishna, J. Maffre, T. Shermer, and O. Roy. CCF: A framework for building confidential verifiable replicated services. Technical Report MSR-TR-2019-16, Microsoft Research, 2019.

[33] N. van Saberhagen. CryptoNote v 2.0. Whitepaper, 2013.

[34] Walmart Inc. Walmart and Sam's Club to require real-time, end-to-end food traceability with blockchain. https://corporate.walmart.com/newsroom/2018/09/24/in-wake-of-romaine-e-coli-scare-walmart-deploys-blockchain-to-track-leafy-greens, 2018.

[35] G. Zyskind, O. Nathan, and A. Pentland. Enigma: Decentralized computation platform with guaranteed privacy. arXiv:1506.03471, 2015.